Retry

eason: 4 · Enterprise · Category: Enterprise · Level: Intermediate · Build with Waqa

Today's Hook

- Today: Retry as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Retry structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Retry
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/enterprise/retry/Retry.java
package com.buildwithwaqar.patterns.enterprise.retry;

import java.util.concurrent.Callable;
import java.util.function.Predicate;

public final class Retry {
    private final int maxAttempts;
    private final Predicate<Exception> retryOn;

    public Retry(int maxAttempts, Predicate<Exception> retryOn) {
        this.maxAttempts = maxAttempts;
        this.retryOn = retryOn;
    }

    public <T> T execute(Callable<T> action) throws Exception {
        Exception last = null;
        for (int i = 1; i <= maxAttempts; i++) {
            try {
                return action.call();
            } catch (Exception e) {
                last = e;
                if (i == maxAttempts || !retryOn.test(e)) throw e;
            }
```

Implementation (2)

```
            }
            throw last;
        }
    }
```

Demo.main

```
package com.buildwithwaqar.patterns.enterprise.retry;
import java.util.concurrent.atomic.AtomicInteger;
public final class Demo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Retry walkthrough ===");
    var n = new AtomicInteger();
    var retry = new Retry(3, e -> e instanceof IllegalStateException);
    var result = retry.execute(() -> {
      int v = n.incrementAndGet();
      System.out.println("attempt=" + v);
      if (v < 3) throw new IllegalStateException("flaky");
      return "published";
    });
    System.out.println("result=" + result);
  }
}
```

Walkthrough Output

```
=== Retry walkthrough ===
attempt=1
attempt=2
attempt=3
result=published
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Retry
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Retry → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.